

LAPORAN PRAKTIKUM WORKSHOP DEVOPS
BAB 4: WEB SERVICE CONTAINER, REVERSE PROXY, DAN ENKRIPSI TLS
(APACHE, NGINX, DAN FLASK API)



Oleh:

Zhulva Priya Abhipraya (3126640034)

Dosen Pengajar:

Dr. Ferry Astika Saputra, ST, M.Sc

PROGRAM STUDI SARJANA TERAPAN LANJUT JENJANG
TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

1. PENDAHULUAN DAN TUJUAN PRAKTIKUM

1.1 Latar Belakang dan Konsep Boundary Aplikasi

Web service dalam arsitektur terkontainerisasi bertindak sebagai titik batas (application boundary) yang menghubungkan pengguna eksternal dengan logika bisnis internal. Pada arsitektur modern berbasis DevSecOps, penataan web service tidak hanya berfokus pada penyajian konten, melainkan pada pembatasan permukaan serangan (attack surface reduction). Menggunakan Nginx sebagai Reverse Proxy di garis depan memungkinkan pemisahan peran secara tegas: Nginx menangani terminasi TLS, pemfilteran request, dan pengalihan trafik (routing), sedangkan layanan backend seperti Apache HTTP Server dan Flask API beroperasi di jaringan tertutup tanpa eksposur port publik secara langsung.

1.2 Perbandingan Arsitektur Nginx vs Apache HTTPD

Dalam pengelolaan web service, memahami perbedaan arsitektur antara Apache HTTPD dan Nginx sangat penting untuk menentukan komponen yang tepat sesuai dengan kebutuhan layanan:

Kriteria Aspek	Apache HTTPD	Nginx
Model Arsitektur	Process / Thread per Request (Prefork/Worker)	Event-Driven Asynchronous (Non-blocking)
Penggunaan Memori & Scaling	Konsumsi RAM membengkak saat trafik tinggi karena alokasi thread baru.	Sangat hemat RAM, mampu melayani puluhan ribu koneksi simultan.
Direktori & Konfigurasi	/usr/local/apache2/conf (Dukungan .htaccess)	/etc/nginx (Konfigurasi terpusat highly optimized)
Peran Utama dalam Lab	Origin Web Server untuk menyajikan halaman statis HTML.	Reverse Proxy, TLS Termination, dan API Gateway Routing.
Reverse Proxy Engine	Modul tambahan (mod_proxy)	Fitur bawaan yang sangat cepat (proxy_pass)

1.3 Konsep Event-Driven Asynchronous pada Nginx

Berbeda dari model tradisional Apache yang menahan satu thread untuk setiap request HTTP yang masuk, Nginx menerapkan arsitektur Event-Driven Asynchronous berbasis Event Loop. Ketika request HTTP diterima, Nginx mendaftarkannya sebagai event I/O non-blocking. Worker process Nginx tidak duduk diam menunggu backend (Flask/Apache) memproses data, melainkan langsung kembali melayani request dari klien lain. Saat backend selesai memproses respons, OS

mengirimkan sinyal pemicu (event trigger/data ready), dan Nginx meneruskan respons HTTP tersebut ke klien. Mekanisme ini memecahkan masalah C10K (Concurrent 10,000 connections) dengan konsumsi CPU dan RAM yang sangat efisien.

1.4 Mekanisme Penyimpanan dan Pengamanan Private Key TLS/SSL

Private key TLS (seperti lab.key) merupakan rahasia kriptografi paling krusial yang digunakan untuk mendekripsi trafik HTTPS. Berdasarkan prinsip DevSecOps, pengamanan private key mengikuti aturan ketat berikut:

1. Larangan Embedding dalam Image/Git: Private key tidak boleh dimasukkan ke dalam Dockerfile, image container, maupun komit repositori Git.
2. Runtime Secret / Read-Only Mount: Key dipasang saat runtime menggunakan bind mount berstatus read-only (:ro) atau Docker Secrets.
3. Minimum File Permission: Mengatur izin akses berkas secara ketat menggunakan chmod 600 certs/lab.key di host, sehingga hanya pemilik/proses berwenang yang dapat membaca file tersebut.

2. ARSITEKTUR LABORATORIUM DAN LINGKUNGAN PRAKTIKUM

2.1 Spesifikasi Lingkungan Kerja

Seluruh rangkaian eksekusi praktikum dijalankan pada lingkungan terisolasi Windows Subsystem for Linux 2 (WSL 2) distro Ubuntu 22.04 LTS di bawah akun pengguna non-root devopsabhi@ASUS. Hal ini menjamin penerapan prinsip least privilege pada sistem operasi host.

Sistem Operasi Host	Windows 11 Home / WSL 2 (Ubuntu 22.04 LTS)
Pengguna / Hostname Linux	devopsabhi@ASUS (Non-root user)
Direktori Kerja Utama	~/docker-lab/bab-4
Engine & Orchestrator	Docker Desktop WSL 2 Integration (Docker Engine 26.x, Compose v2)
Port Mappings Host	127.0.0.1:8084 (HTTP Redirect) & 127.0.0.1:8443 (HTTPS TLS)
Internal Bridge Network	web-net (Driver: bridge, Isolated backend)

2.2 Arsitektur Multi-Container dan Topologi Jaringan

Arsitektur Bab 4 terdiri dari 3 service utama yang saling terhubung dalam jaringan internal web-net. Nginx bertindak sebagai satu-satunya pintu masuk (Single Point of Entry) yang menerima request publik, melakukan verifikasi TLS, lalu membaginya berdasarkan URI path:

1. **Routing Web Statis (/):** Path / dialihkan ke service apache-web pada port internal 80 untuk menyajikan halaman HTML statis.

2. Routing API Dinamis (/api/): Path /api/ dialihkan ke service flask-app pada port internal 5000 untuk menyajikan respons JSON dinamis.

3. LANGKAH-LANGKAH PRAKTIKUM DAN EKSEKUSI TEKNIS

3.1 Inisialisasi Struktur Direktori Kerja

Langkah awal diawali dengan pembuatan struktur folder terorganisir di bawah direktori ~/docker-lab/bab-4 sebagai berikut:

```
devopsabhi@ASUS:~$ mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
devopsabhi@ASUS:~$ ls
docker-lab
devopsabhi@ASUS:~$ cd docker-lab
devopsabhi@ASUS:~/docker-lab$ cd bab-4
devopsabhi@ASUS:~/docker-lab/bab-4$ ls
apache app certs logs nginx
devopsabhi@ASUS:~/docker-lab/bab-4$
```

3.2 Generasi Sertifikat Digital TLS Self-Signed

Sertifikat kriptografi TLS dibuat menggunakan OpenSSL dengan kunci RSA 2048-bit dan masa berlaku 365 hari. Sertifikat dilengkapi ekstensi Subject Alternative Name (SAN) agar dikenali valid untuk localhost dan IP 127.0.0.1:

[illegible]

3.3 Pengembangan Service Backend (Apache & Flask Gunicorn Non-Root)

Aplikasi web statis disiapkan pada `apache/sites/index.html` dengan konten HTML kustom:

```
GNU nano 8.7.1                                compose.yaml
services:|
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy

  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net
```

Selanjutnya, dikembangkan aplikasi Flask API dinamis pada app/app.py beserta berkas app/requirements.txt:

```
GNU nano 8.7.1                                nginx/conf/default.conf
upstream apache_backend {
    server apache-web:80;
}

upstream flask_backend {
    server flask-app:5000;
}

server {
    listen 80;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}

server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate      /etc/nginx/certs/lab.crt;
    ssl_certificate_key  /etc/nginx/certs/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;
```

Penerapan DevSecOps Hardening diwujudkan pada app/Dockerfile dengan membuat pengguna sistem non-root (appuser, UID 10001):

```
devopsabhi@ASUS: ~/docker X + v
GNU nano 8.7.1 apache/sites/index.html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4 Zhulva</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
```

```
Flask==3.1.2
gunicorn==23.0.0
```

```
GNU nano 8.7.1 app/app.py
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )

@app.get("/health")
def health():
    return jsonify(status="healthy"), 200
```

3.4 Konfigurasi Nginx Reverse Proxy dan Redireksi HTTPS

Konfigurasi Nginx ditulis pada `nginx/conf/default.conf` untuk menangani TLS termination, redireksi HTTP 301, pencatatan header terpercaya, serta pengalihan lokasi URI:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )

@app.get("/health")
def health():
    return jsonify(status="healthy"), 200
```

3.5 Orkestrasi Multi-Container via Docker Compose (compose.yaml)

Seluruh komponen diorkestrasikan secara deklaratif dalam berkas `compose.yaml`. Terjadi penyesuaian pemetaan port host dari 8080 menjadi 8084 untuk menghindari port collision pada sistem Windows/WSL2:

```
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8084:80"
      - "127.0.0.1:8443:443"
    volumes:
```

4. ANALISIS HASIL DAN TROUBLESHOOTING KENDALA OPERASIONAL

4.1 Verifikasi Status Stack dan Pengujian Konektivitas

Setelah mengeksekusi `docker compose up -d --build`, status kontainer diperiksa melalui `docker compose ps`:

```
=> [flask-app] exporting to image 3.4s
=> => exporting layers 1.9s
=> => exporting manifest sha256:e672c5fe72a0112aef2462e201bc6ae535945f5266ac4527226252982364b1ab 0.0s
=> => exporting config sha256:81f357d62efe12a0b0dc1dbd6bd3aeb927613895ec91016333b99b13ba899781 0.0s
=> => exporting attestation manifest sha256:0da880236beela59d5dc1df8fetc750a89fa6c23d366d37201769b170b0cbf4e 0.1s
=> => exporting manifest list sha256:b927aecf5354e352eee560d6b88bea6fdf1230c766c0e2901b3b9aad5e67ed98 0.0s
=> => naming to docker.io/library/bab-4-flask-app:latest 0.0s
=> => unpacking to docker.io/library/bab-4-flask-app:latest 1.2s
=> [flask-app] resolving provenance for metadata file 0.1s
[+] Running 4/5
✔flask-app Built 0.0s
✔Network bab-4_web-net Created 0.2s
✔Container bab-4-flask-app-1 Healthy 8.1s
✔Container bab-4-apache-web-1 Started 2.6s
! Container bab-4-proxy-1 Starting 8.0s
Error response from daemon: Ports are not available: exposing port TCP 127.0.0.1:8080 -> 127.0.0.1:0: listen tcp 127.0.0.1:8080: bind: An attempt was made to access a socket in a way forbidden by its access permissions.
devopsabhi@ASUS:~/docker-lab/bab-4$ nano compose.yaml
devopsabhi@ASUS:~/docker-lab/bab-4$ docker compose up -d
[+] Running 3/3
✔Container bab-4-apache-web-1 Running 0.0s
✔Container bab-4-flask-app-1 Healthy 2.2s
✔Container bab-4-proxy-1 Started 2.8s
devopsabhi@ASUS:~/docker-lab/bab-4$ |
```

Selanjutnya dilakukan tiga tahap pengujian konektivitas menggunakan `curl`:

a. Pengujian Redireksi HTTP ke HTTPS (Port 8084): Mengembalikan respons HTTP/1.1 301 Moved Permanently yang mengarahkan trafik ke lokasi `https://localhost:8443/`.

b. Pengujian Akses HTTPS Apache (Port 8443): Mengembalikan respons HTTP/1.1 200 OK beserta seluruh Security Headers dan konten HTML Apache statis.

```
devopsabhi@ASUS:~/docker-lab/bab-4$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
bab-4-apache-web-1  httpd:2.4-alpine    "httpd-foreground"    apache-web 2 minutes ago  Up 2 minutes  80/tcp
bab-4-flask-app-1   bab-4-flask-app     "gunicorn --bind 0.0..." flask-app  2 minutes ago  Up 2 minutes (healthy)  5000/tcp
bab-4-proxy-1       nginx:alpine        "/docker-entrypoint..." proxy      About a minute ago  Up About a minute  127.0.0.1:8084->80/tcp, 127.0.0.1:8443->443/tcp
devopsabhi@ASUS:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 29 Sep 2026 11:58:12 GMT
Content-Type: text/html
Content-Length: 289
Connection: keep-alive
Last-Modified: Tue, 29 Sep 2026 11:42:05 GMT
ETag: "121-65c9dabfc6384"
Accept-Ranges: bytes
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4 Zhulva</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
```

c. Pengujian Akses Routing Flask API (/api/): Mengembalikan respons JSON dari Flask API yang mengonfirmasi penerimaan header `X-Forwarded-Proto`: `https`.

```
</html>
devopsabhi@ASUS:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}
devopsabhi@ASUS:~/docker-lab/bab-4$ |
```

4.2 Problem Solving Kendala Operasional Real-World

Selama eksekusi praktikum, terjadi dua kendala teknis utama yang berhasil dianalisis dan diselesaikan secara sistematis:

1. Masalah Port Collision & Socket Permission (Port 8080 -> 8084): Saat menjalankan docker compose up -d pertama kali, muncul pesan error: 'Ports are not available: listen tcp 127.0.0.1:8080: bind: An attempt was made to access a socket in a way forbidden by its access permissions'.

Secara teknis, hal ini disebabkan oleh port 8080 host yang termasuk dalam reserved port range oleh sistem Hyper-V / WinNAT Windows. Solusi dilakukan dengan mengubah pemetaan port HTTP pada compose.yaml menjadi 127.0.0.1:8084:80. Setelah perubahan port, kontainer proxy dapat melakukan socket binding secara sempurna.

2. Masalah Terputusnya WSL 2 Integration Docker Desktop: Terjadi pesan kesalahan 'The command docker could not be found in this WSL 2 distro'. Hal ini disebabkan oleh terputusnya integrasi daemon pasca restart WSL. Solusi dilakukan dengan membuka Docker Desktop Settings -> Resources -> WSL Integration, mengaktifkan sakelar toggle untuk distro Ubuntu, lalu mengeksekusi Apply & restart.

5. EVALUASI KEAMANAN (DEVSECOPS) DAN KESIMPULAN

5.1 Evaluasi Keamanan Infrastruktur Web

Implementasi praktikum Bab 4 telah memenuhi pilar dasar keamanan DevSecOps:

1. Hardening Pengguna Kontainer: Penggunaan pengguna non-root (appuser, UID 10001) pada kontainer Flask mencegah potensi privilege escalation ke host jika terjadi kerentanan pada aplikasi.

2. Immutable Volume Mount: Penggunaan instruksi :ro (read-only) pada volume konfigurasi Nginx dan situs Apache mencegah modifikasi berkas oleh pihak tidak berwenang saat runtime.

3. HTTP Security Headers: Pemasangan header X-Content-Type-Options, X-Frame-Options, dan Referrer-Policy secara konsisten melindungi klien dari serangan MIME-sniffing dan Clickjacking.

5.2 Kesimpulan Praktikum

Berdasarkan seluruh hasil eksperimen dan pengujian yang dilakukan, dapat disimpulkan bahwa:

1. Nginx berhasil diimplementasikan sebagai Reverse Proxy tunggal yang mengendalikan lalu lintas HTTP/HTTPS dan membaginya secara tepat ke backend Apache (statis) dan Flask (dinamis).
2. Penerapan TLS termination pada Nginx terbukti mampu mengamankan komunikasi data dari luar (HTTPS) tanpa membebani pemrosesan pada aplikasi backend.
3. Penggunaan Docker Compose dengan fitur healthcheck dan kustomisasi port binding (8084) mampu menghadirkan infrastruktur mikroservis yang tangguh, terisolasi, dan mudah dikelola.